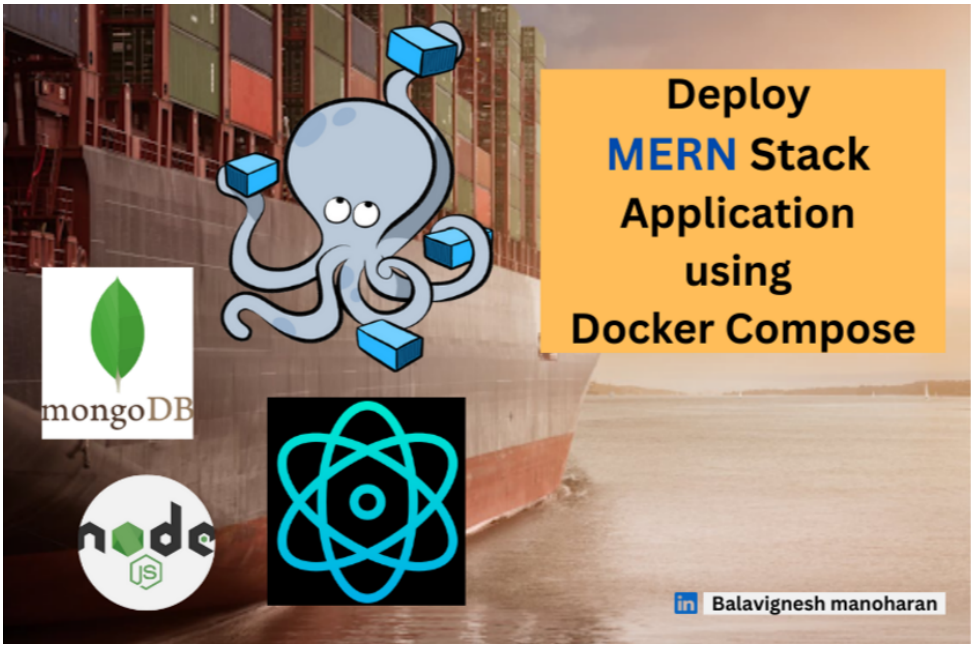
# Deploy MERN Stack Application using Docker Compose




## Introduction:

In this article, we will containerize a **MERN** Stack application using **Docker** and then later deploy it with **Docker Compose**. We will explore what is a MERN Stack application is, why it is popular,  and the different environments which is required for this application. We will then build each environment using docker and run each application in separate containers, then finally deploy the entire stack using a single YAML file with Docker Compose.

Before knowing what is a MERN Stack application is, let's understand what is a Three-Tier-Architecture.

**In simple terms, Three-Tier-Application means Three Main Layers.**

1. **Presentation Layer**: This is the UI or Frontend, where the users will actually interact with the application.
2. **Business logic**: This is the Backend layer, where all the data processing happens.
3. **Data Layer**: This is the Database layer or storage where all the data is stored and fetched from.

This is typically how a Three-Tier-Architecture looks and functions.

## What is MERN Stack ?

MERN Stands for

- M: MongoDB (Data Layer)
- E: Express.js (Business logic layer)
- R: React.js (Presentation Layer)
- N: Node.js (Backend or Server)

Together, these technologies can create a scalable application. What makes MERN Stack so popular is that all the technologies mentioned above are **Open Source** and most come from the **JavaScript family**. This is easy for developers to understand and develop code from front-end to back-end.

As Discussed, in this application we have three layers. First we will containerize each layer separately, then we will deploy the entire stack of different layers together using a YAML file which will be our Docker Compose.

## Pre-requisites:

- Fork/clone or download the code to your local
- Docker

## Steps:

1. Clone the project to your local.

```
git clone https://github.com/balavi7/MERN-Docker-Compose.git
```

2. In this application, we will create Dockerfile for frontend and backend, and database we will deploy directly using Mongo image.
3. So, to create a Dockerfile for the MERN application for frontend copy the below code

```
#Using the Offcial Base image
FROM node:18.9.1 as build

#Create a Work directory
WORKDIR /app

#Copy package.json files from host to the container
COPY package*.json .

#Install the dependencies
RUN npm install

#Copy rest of the files inside the container
COPY . .

#List the port in which the application is running
EXPOSE 5173

#Run the application
CMD ["npm", "run", "dev"]
```

4. After you've create Dockerfile, let's build this dockerfile and create an image.

```
docker build -t mern-frontend .
```

```
bala_pc@Bala:~/projects/MERN-docker-compose/mern/frontend$ docker images | head
REPOSITORY                    TAG       IMAGE ID       CREATED          SIZE
mern-frontend                 latest    e92ed7ca7b05   22 minutes ago   1.7GB
```

5. Now we need to create a network and use the same network for all the containers.

> Why to create a Network?
> Within a same Docker network, containers can communicate with each other more easily. Also it enhances security and communication, making the environment easy to manage.

```
docker network create mern
```

```
bala_pc@Bala:~/projects/MERN-docker-compose/mern/frontend$ docker network create mern
ca0bb6f0b413783e8cc14d459647bc6f230c9154e47e0d8053a8636ed06a9468
```

6. Now run the frontend container to check if it is running.

```
docker run -d -p 5173:5173 --name=frontend --network=mern mern-f
```

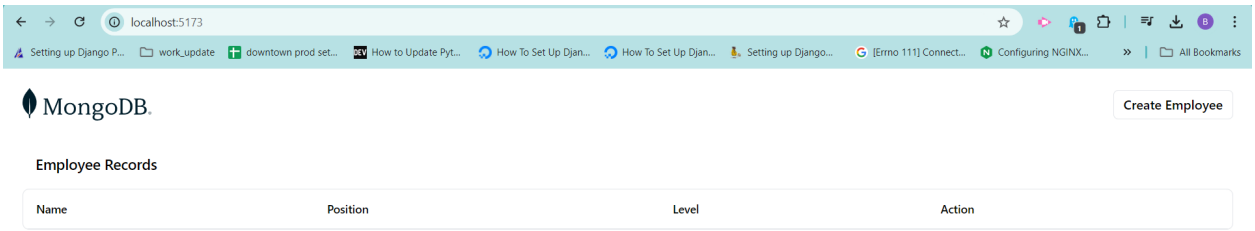
7. We will check if the container is running.

```
docker ps
```

```
bala_pc@Bala:~/projects/MERN-docker-compose/mern/frontend$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED          STATUS         PORTS                    NAMES
f8d012328a61   mern-frontend  "docker-entrypoint.s…"   9 seconds ago    Up 3 seconds   0.0.0.0:5173->5173/tcp   frontend
```

8. Open your browser and run it using localhost

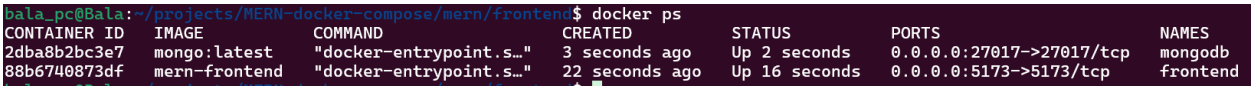
```
http://localhost:5173/
```

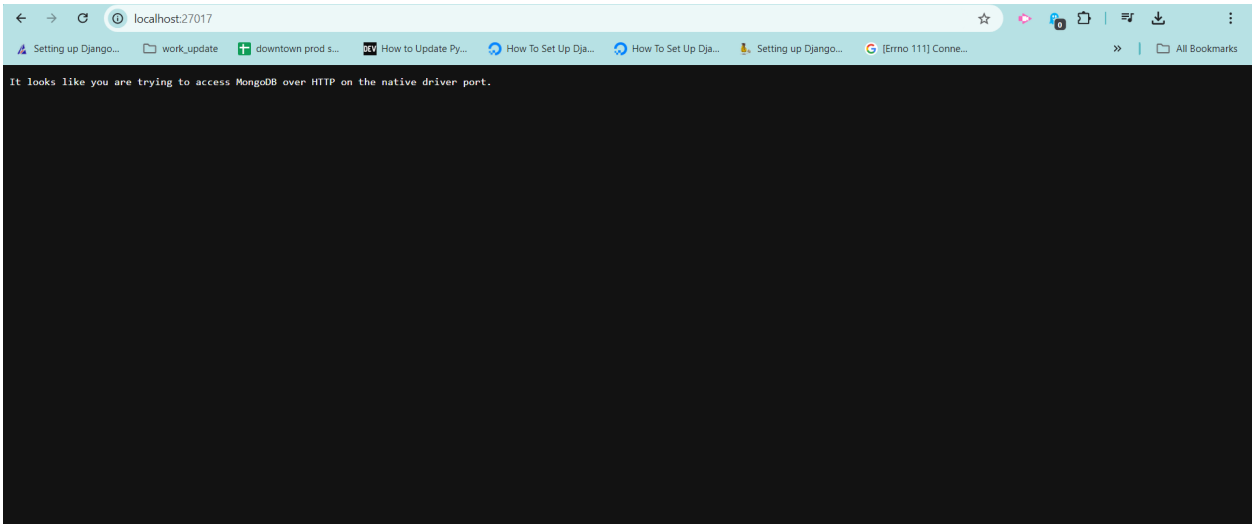



9. As you can see, frontend is running. So let's create deploy backend as well in the next step.
10. Before deploying backend, let's run the mongodb.
11. We are running mongodb directly by using mongo latest image, so run the below docker command to start the mongo container.

```
docker run -d -p 27017:27017 --name=mongodb --network=mern -v ~/
go:latest
```

12. Check if the mongodb container is running.



13. Now it's time for backend, let us create a Dockerfile for the backend.

```
#Using the Offcial Base image
FROM node:18.9.1

#Create a Work directory
WORKDIR /app

#Copy package.json files from host to the container
COPY package.json .

#Install dependencies
RUN npm install

#Copy rest of the files inside the container
COPY . .

#List the port in which the application is running
EXPOSE 5050

#Run the application
CMD ["npm", "start"]
```

14. Once the Dockerfile is created, build the image

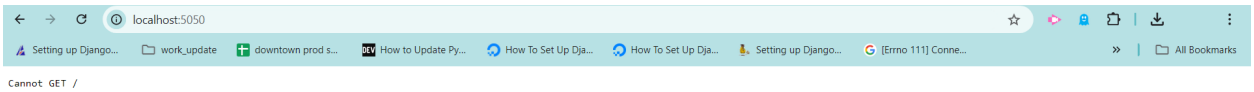
```
docker build -t mern-backend .
```

15. Now let us run the backend container using the below code.

```
docker run -d -p 5050:5050 --network=mern --name=backend mern-ba
```

16. Check if the backend container is running.

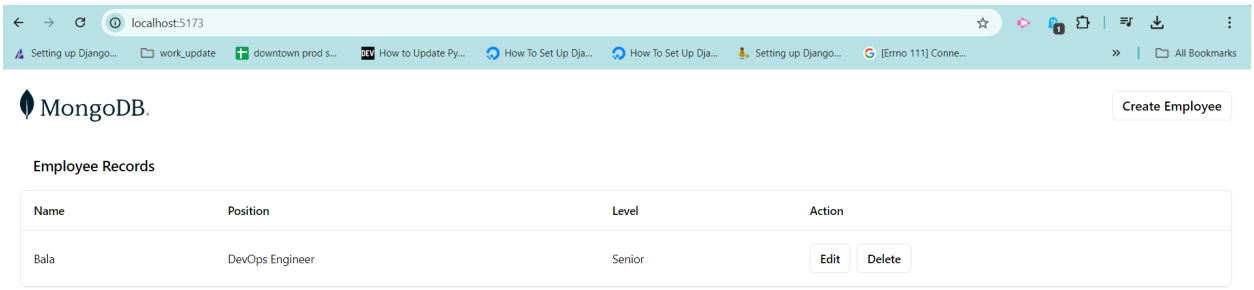
```
bala_pc@Bala:~/projects/MERN-docker-compose/mern/backend$ docker ps
CONTAINER ID   IMAGE           COMMAND                  CREATED          STATUS          PORTS                     NAMES
46a449fd28f5   mern-backend    "docker-entrypoint.s…"   16 seconds ago   Up 8 seconds    0.0.0.0:5050->5050/tcp    backend
2dba8b2bc3e7   mongo:latest    "docker-entrypoint.s…"   11 minutes ago   Up 11 minutes   0.0.0.0:27017->27017/tcp  mongodb
88b6740873df   mern-frontend   "docker-entrypoint.s…"   11 minutes ago   Up 11 minutes   0.0.0.0:5173->5173/tcp    frontend
```




```
Cannot GET /
```

17. Now all the containers are running, let us check if our application is running fine, and are we able to save the employee record and edit is as well.
18. Go to the frontend and try to save an employee record.

```
http://localhost:5173/
```




Voila! It's Working. Now we are able to save the record and you can edit it as well.

19. Instead of running multiple commands and executing numerous steps, you can deploy everything with a single command. How is this possible? We will use a YAML file, specifically the Docker Compose file.
20. Now stop all the containers and delete all the containers

```
docker rm -f 46a449fd28f5 2dba8b2bc3e7 88b6740873df
```

```
bala_pc@Bala:~/projects/MERN-docker-compose/mern$ docker rm -f 46a449fd28f5 2dba8b2bc3e7 88b6740873df
46a449fd28f5
2dba8b2bc3e7
88b6740873df
```

21. Check if any container is running.

```
docker ps
```

```
bala_pc@Bala:~/projects/MERN-docker-compose/mern$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
```

22. At the root level, let us create a file called docker compose. This is the file with which we will run all the containers using a single command.

```
services:
  backend:
    build: ./mern/backend
    ports:
      - "5050:5050"
    networks:
      - mern
    depends_on:
      - mongodb

  frontend:
    build: ./mern/frontend
    ports:
      - "5173:5173"
    networks:
      - mern

  mongodb:
    image: mongo:latest
```

```
    ports:
      - "27017:27017"
    networks:
      - mern
    volumes:
      - mongo-data:/data/db

networks:
  mern_network:
    driver: bridge

volumes:
  mongo-data:
    driver: local
```

23. Save this file, and the only command you need to run is given as below.

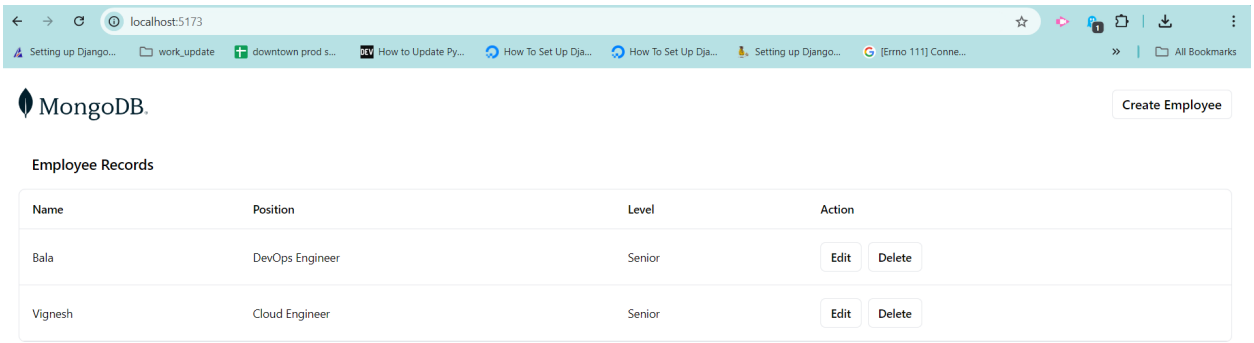
```
docker compose up -d
```

```
[+] Running 3/5
 ⠿ Network mern-docker-compose_mern          Created
 ⠿ Volume "mern-docker-compose_mongo-data"   Created
 ✔ Container mern-docker-compose-frontend-1  Started
 ✔ Container mern-docker-compose-mongodb-1   Started
 ✔ Container mern-docker-compose-backend-1   Started
```

As you can see, it has created all the services mentioned inside the compose file and also created the network and volume.

24. Let us check if the site is working.

```
http://localhost:5173/
```




**Success** ! Our application is working using docker compose as well.

## Conclusion:

Dockerizing a MERN Stack application right from setting up individual containers, creating network and managing all the containers in a separate isolated network. And in the later stage, effortlessly managing everything with Docker Compose.

If you found this post useful, give it a like👍

Repost♻️

Follow @Bala Vignesh  for more such posts 💯🚀